

Module 10 — Evaluation Metrics

src/evaluation_metrics.py • 198 lines

In one sentence

The ranking metrics, written from scratch rather than imported.

1. What this module does

Measuring a recommender is not like measuring a classifier. What matters is not just whether the right products appear, but **where in the list** they appear - because customers click the top of a list far more than the bottom.

This module implements the four metrics the business case requires, plus four more that measure things accuracy cannot see: how much of the catalogue is ever recommended, how varied a single list is, and how obvious the suggestions are.

2. Concepts covered

Every idea this module uses, in plain terms.

Concept	In simple terms	Where it is used here
Precision@K	Of K shown, how many were right	<code>precision_at_k()</code>
Recall@K	Of all right answers, how many did we show	<code>recall_at_k()</code>
MAP@K	Rewards putting hits near the top	<code>average_precision_at_k()</code>
NDCG@K	Position-weighted score - the headline metric	<code>ndcg_at_k()</code>
Discounted cumulative gain	A hit at position 1 beats one at position 10	The log2 discount
Graded relevance	A 5-star hit is worth more than a 4-star	Optional <code>relevance_scores</code>
Hit rate	Did we get at least one right?	<code>hit_rate_at_k()</code>
Catalogue coverage	How much stock ever gets shown	<code>catalogue_coverage()</code>
Novelty	How non-obvious the suggestions are	<code>novelty()</code> - uses self-information
Intra-list diversity	Are the 10 results all the same thing?	<code>intra_list_diversity()</code>
Beyond-accuracy evaluation	Metrics accuracy cannot capture	Why the last four exist

3. How it works, step by step

1. **Precision@K** counts hits in the top K and divides by how many were actually returned - not by K, so a model is not punished for items it could never produce.
2. **Recall@K** divides hits by the total number of relevant items that existed.
3. **MAP@K** recalculates precision at each hit position and averages, so hits near the top score higher.
4. **NDCG@K** divides each hit's value by log2 of its position, then divides by the best possible arrangement.
5. **Coverage** collects every product ever recommended to anyone and divides by the catalogue size.
6. **Novelty** averages -log2 of each product's probability, so obvious best-sellers score low.

4. Key functions

Function	What it measures
precision_at_k()	Accuracy of the shown list
recall_at_k()	Completeness against everything relevant
average_precision_at_k()	Accuracy weighted toward the top
ndcg_at_k()	Position-discounted quality - the headline
hit_rate_at_k()	Did we get anything right at all
catalogue_coverage()	Share of stock ever recommended
long_tail_share()	Share of results from the tail
intra_list_diversity()	Variety within one list
novelty()	How non-obvious the results are
evaluate_ranking()	Runs everything and averages over customers

5. Inputs, outputs and how to run

	Detail
Inputs	Recommendation lists and the true relevant sets
Outputs	A DataFrame of metrics at each K
Used by	notebooks/recommendation_evaluation.py

6. Key code

src/evaluation_metrics.py — ndcg_at_k()

```

51 | def ndcg_at_k(recommended: list, relevant: set, k: int,
52 |             relevance_scores: dict = None) -> float:
53 |     """Normalised discounted cumulative gain, with optional graded relevance."""
54 |     if not recommended or not relevant:
55 |         return 0.0
56 |
57 |     def gain(item) -> float:
58 |         if relevance_scores is None:
59 |             return 1.0 if item in relevant else 0.0
60 |         return float(relevance_scores.get(item, 0.0))
61 |
62 |     dcg = sum(
63 |         gain(item) / np.log2(position + 1)
64 |         for position, item in enumerate(recommended[:k], start=1)
65 |         if item in relevant
66 |     )
67 |
68 |     if relevance_scores is None:
69 |         ideal_gains = [1.0] * min(len(relevant), k)
70 |     else:
71 |         ideal_gains = sorted(
72 |             (relevance_scores.get(i, 0.0) for i in relevant), reverse=True
73 |         )[:k]
74 |
75 |     idcg = sum(g / np.log2(position + 1) for position, g in enumerate(ideal_gains, start=1))
76 |
77 |     return dcg / idcg if idcg > 0 else 0.0

```

The most important metric in the project.

src/evaluation_metrics.py — catalogue_coverage()

```

90 | def catalogue_coverage(all_recommendations: list, total_items: int) -> float:
91 |     """Share of the catalogue that ever appears in any recommendation."""
92 |     if total_items == 0:
93 |         return 0.0
94 |
95 |     recommended_items = set()
96 |     for recommendations in all_recommendations:
97 |         recommended_items.update(recommendations)
98 |
99 |     return len(recommended_items) / total_items

```

7. Things to remember

Why NDCG rather than plain precision

Precision@10 treats a hit at position 1 and a hit at position 10 as equally good. On a real storefront they are worth very different amounts, because click-through falls sharply down the page. NDCG divides each hit by log2 of its position, so it tracks actual revenue much more closely.

Why coverage is not optional

A model can post excellent precision while only ever recommending 32 products out of 2,500. Commercially that is a failure - the rest of the inventory is invisible and will never sell. Accuracy metrics cannot detect this at all. In this project coverage is exactly what exposed the broken NCF, which had 0.6% coverage.

A subtlety in MAP

It divides by min(number of relevant items, K), not by the number of hits. Dividing by hits alone would award a perfect 1.0 to a model that found exactly one relevant item and put it first - which is clearly not perfect performance.

8. Likely questions

Q. Explain NDCG in simple terms.

A. It scores a list by how many relevant items it contains and how near the top they are. Each hit is worth 1 divided by $\log_2$ of its position, so position 1 is worth much more than position 10 . That total is then divided by the score of the best possible ordering, giving a number between 0 and 1 .

Q. Why is Recall@10 low even for the best model?

A. There is a mathematical ceiling. If a customer has 40 relevant products in the test period, Recall@10 cannot exceed 0.25 no matter how perfect the model is. Recall always has to be read alongside how many relevant items each customer actually had - here the average is 4.4 .

Q. Why did you write these instead of using a library?

A. The business case asks for custom metric implementations, and the edge cases matter. How you handle a customer with no relevant items, or K larger than the list returned, changes the numbers - writing them means I know exactly what is being measured.